

APOSTILA: 5 PRINCÍPIOS SOLID IMPLEMENTADOS EM JAVA

Princípios SOLID

Prof. Marlon

SOLID não é uma tecnologia nova nem uma palavra-chave do Java. É um conjunto de princípios que nos ajuda a organizar melhor o código utilizando conceitos que já conhecemos: classes, herança, interfaces, polimorfismo e injeção de dependência.

Objetivos da apostila

- Compreender os cinco princípios SOLID
- Relacionar SOLID com os conteúdos já estudados
- Identificar problemas de acoplamento
- Utilizar interfaces para reduzir dependências
- Produzir código mais flexível e reutilizável

Base de conhecimento requerida

- Associação entre objetos
- Herança
- Polimorfismo
- Interfaces
- Tratamento de erros
- Injeção de Dependência

Esses conceitos funcionam bem em sistemas pequenos.

Porém, em sistemas maiores, surge um problema:

*Quanto mais o sistema cresce,
mais difícil se torna modificá-lo.*

SOLID é um conjunto de princípios criado para tornar o código:

- Mais organizado
- Mais flexível
- Mais fácil de manter
- Mais fácil de testar

O que significa SOLID?

S = Single Responsibility Principle

O = Open/Closed Principle

L = Liskov Substitution Principle

I = Interface Segregation Principle

D = Dependency Inversion Principle

S - Single Responsibility Principle (SRP)

Uma classe deve ter apenas uma responsabilidade

Problema

```
public class Usuario {  
    public void cadastrar() { }  
    public void enviarEmail() { }  
    public void gerarRelatorio() { }  
}
```

Pergunta: Quantos motivos existem para alterar essa classe?

Resposta: Cadastro, E-mail, Relatório, Muitos motivos.

Solução

```
public class UsuarioService {  
  
    public void cadastrar() { }  
  
}  
public class EmailService {  
  
    public void enviarEmail() { }  
  
}  
public class RelatorioService {  
  
    public void gerarRelatorio() { }  
  
}
```

Cada classe possui apenas uma responsabilidade.

Ligação com DI (Dependency Injection)

Com responsabilidades separadas fica mais fácil injetar dependências.

```
public class PedidoService {  
    private EmailService email;  
}
```

O - Open/Closed Principle (OCP)

- Aberto para extensão
- Fechado para modificação

Sem interfaces

```
public class PedidoService {  
  
    public void enviarNotificacao() {  
        Email email = new Email();  
        email.enviar();  
    }  
  
}
```

Se aparecer SMS:

```
public class PedidoService {  
  
    public void enviarNotificacao() {  
  
        SMS sms = new SMS();  
  
        sms.enviar();  
    }  
  
}
```

Precisamos modificar a classe.

Utilizando interface

```
public interface Notificavel {  
    void enviar();  
}
```

```

    }
    public class Email implements Notificavel {
        public void enviar() { }
    }
    public class SMS implements Notificavel {
        public void enviar() { }
    }
    public class PedidoService {
        private Notificavel notificacao;
    }

```

Agora podemos criar novas implementações sem alterar a classe principal.

Ligação com Interfaces

OCP funciona graças ao uso de:

- Interfaces
- Polimorfismo

L - Liskov Substitution Principle (LSP)

Uma classe filha deve poder substituir a classe pai

Exemplo correto:

```

public class Funcionario {

    public double calcularBonus() {
        return 0;
    }

}

public class Gerente
    extends Funcionario {

    @Override
    public double calcularBonus() {
        return 1000;
    }

}

Funcionario f = new Gerente();

```

Funciona normalmente.

Lembrando herança

```
Funcionario f =  
    new Desenvolvedor();  
  
Funcionario f = new Gerente();
```

Isso é exatamente o princípio de Liskov.

A classe filha pode substituir a pai sem quebrar o programa.

I - Interface Segregation Principle (ISP)

Não obrigue classes a implementar métodos que não utilizam

Interface grande demais

```
public interface Trabalhador {  
  
    void programar();  
  
    void testar();  
  
    void vender();  
  
    void contratar();  
  
}
```

Problema

Um desenvolvedor não vende.

```
public class Desenvolvedor  
    implements Trabalhador {  
  
}
```

Será obrigado a implementar métodos sem sentido.

Solução: Interfaces menores

```

public interface Programador {

    void programar();

}
public interface Testador {

    void testar();

}
public interface Vendedor {

    void vender();

}

```

Agora cada classe implementa apenas o que precisa.

Ligação com Interfaces

Nem toda interface é uma boa interface. Uma interface deve representar um contrato pequeno e coeso.

D - Dependency Inversion Principle (DIP)

- Dependenda de abstrações
- Não dependa de implementações

Exemplo errado

```

public class Usuario {

    public void cadastrar() { }

    public void enviarEmail() { }

    public void gerarRelatorio() { }

}

```

Pergunta: Quantos motivos existem para alterar essa classe?

Resposta: Cadastro, E-mail, Relatório, Muitos motivos.

Solução

```
public class UsuarioService {  
  
    public void cadastrar() { }  
  
}  
public class EmailService {  
  
    public void enviarEmail() { }  
  
}  
..  
public class RelatorioService {  
  
    public void gerarRelatorio() { }  
  
}
```

Cada classe possui apenas uma responsabilidade.

Ligação com DI

Com responsabilidades separadas fica mais fácil injetar dependências.

```
public class PedidoService {  
  
    private EmailService email;  
  
}
```

A Injeção de Dependência é uma forma prática de aplicar DIP.

DIP = princípio

DI = técnica para implementar o princípio

Essa conexão costuma cair bastante em entrevistas.

Resumo Geral

Princípio	Ideia Principal
SRP	Uma responsabilidade por classe
OCP	Estender sem modificar
LSP	Substituir a classe pai sem problemas

ISP	Interfaces pequenas e específicas
DIP	Depender de interfaces e não de implementações

Exercício

Considere o sistema abaixo:

```
public class Loja {

    private Email email = new Email();

}
```

Identifique:

1. Qual princípio está sendo violado?
2. Como utilizar interface?
3. Como aplicar Injeção de Dependência?
4. Qual implementação poderia substituir Email?

Exercício em Equipe

Modelar um sistema de:

- Biblioteca
- Academia
- Clínica
- E-commerce
- Hotel

Identificar no projeto:

1. Um exemplo de SRP
2. Um exemplo de OCP
3. Um exemplo de LSP
4. Um exemplo de ISP
5. Um exemplo de DIP

Se sobrar tempo, implemente os princípios

Fechamento

- Associação -> relaciona objetos
- Herança -> reutiliza código
- Polimorfismo -> permite trocar comportamentos
- Interfaces -> definem contratos
- DI -> reduz acoplamento
- SOLID -> organiza tudo isso
 - para construir sistemas mais flexíveis e fáceis de manter.

Essa abordagem costuma funcionar muito bem porque os alunos percebem que SOLID não é um assunto novo, mas uma consequência natural de tudo o que já aprenderam nas aulas anteriores.